

Experiment #03

Creating Logical C++ codes using Functions and Arrays concepts

Objective

The objective of this lab session is to reinforce some basic programming concepts regarding function, types of functions, define user functions and arrays in C++.

Software Tools

Dev C++

Theory

Function

A function is a subprogram that acts on data and often returns a value. A program written with numerous functions is easier to maintain, update and debug than one very long program. By programming in a modular (functional) fashion, several programmers can work independently on separate functions which can be assembled at a later date to create the entire project. Each function has its own name. When the function is finished, execution returns to the area of the program code from which it was called, and the program continues on to the next line of code.

Steps for creating user-defined functions

Declare the function

The declaration, called the **FUNCTION PROTOTYPE**, informs the compiler about the functions to be used in a program, the argument they take and the type of value they return.

Define the function

The function definition tells the compiler what task the function will be performing. The function prototype and the function definition must be same on the return type, the name, and the parameters. The only difference between the function prototype and the function header is a semicolon.

The function definition consists of the function header and its body. The header is EXACTLY like the function prototype, EXCEPT that it contains NO terminating semicolon.

//Prototyping, defining and calling a function

```
#include <iostream>
using namespace std;
void func();      // prototype the function
int main()
{
    func( );      // function call
    cout<< "\t\tBjarne Stroustrup\n";
    func( );      // function call
    return 0;
}
```

```

    // function definition
void func()
{
    int count;    // declaring a LOCAL variable
    for(count = 1; count <=65; count++)
        cout<< "*";
    cout<<endl;
}

```

Argument to a function

Sometimes the calling function supplies some values to the called function. These are known as parameters. The variables which supply the values to a calling function called **actual parameters**. The variable which receive the value from called statement are termed **formal parameters**.

Passing int type argument to a function

```

#include<iostream>
using namespace std;
void age(int);
int main()
{
    int a;
    cout<<"enter two ages:";
    cin>>a>>b;
    age(a);
    return 0;
}
void area(int b)
{
    cout<< "sum of ages is"<<b+b<<"\n";
}

```

Passing float type argument to a function

```

#include<iostream>
using namespace std;
void area(float);
int main()
{
    float radius;
    cin>>radius;
    area(radius);
    return 0;
}
void area(float r)
{

```

```
    cout<< "the area of the circle is"<<3.14*r*r<<"\n";
}
```

Here radius is called **actual parameter** and r is called **formal parameter**.

Return type of a function

// Example program

```
#include <iostream>
```

```
using namespace std;
```

```
int timesTwo(int num); // function prototype
```

```
int main()
```

```
{
```

```
    int number, response;
```

```
    cout<<"Please enter a number:";
```

```
    cin>>number;
```

```
    response = timesTwo(number);    //function call
```

```
    cout<< "The answer is "<<response;
```

```
    return 0;
```

```
}
```

```
//timesTwo function
```

```
int timesTwo (int num)
```

```
{
```

```
    int answer;        //local variable
```

```
    answer = 2 * num;
```

```
    return (answer);
```

```
}
```

Function with default arguments

C++ allows calling a function without specifying all its arguments. In such cases, the function assigns a default value to a parameter that does not have a matching argument in the function call. Default values are specified when the function is declared. The compiler knows from the prototype how many arguments a function uses for calling.

Example

```
float result(int marks1, int marks2, int marks3=75);
```

a subsequent function call

```
average = result(60,70);
```

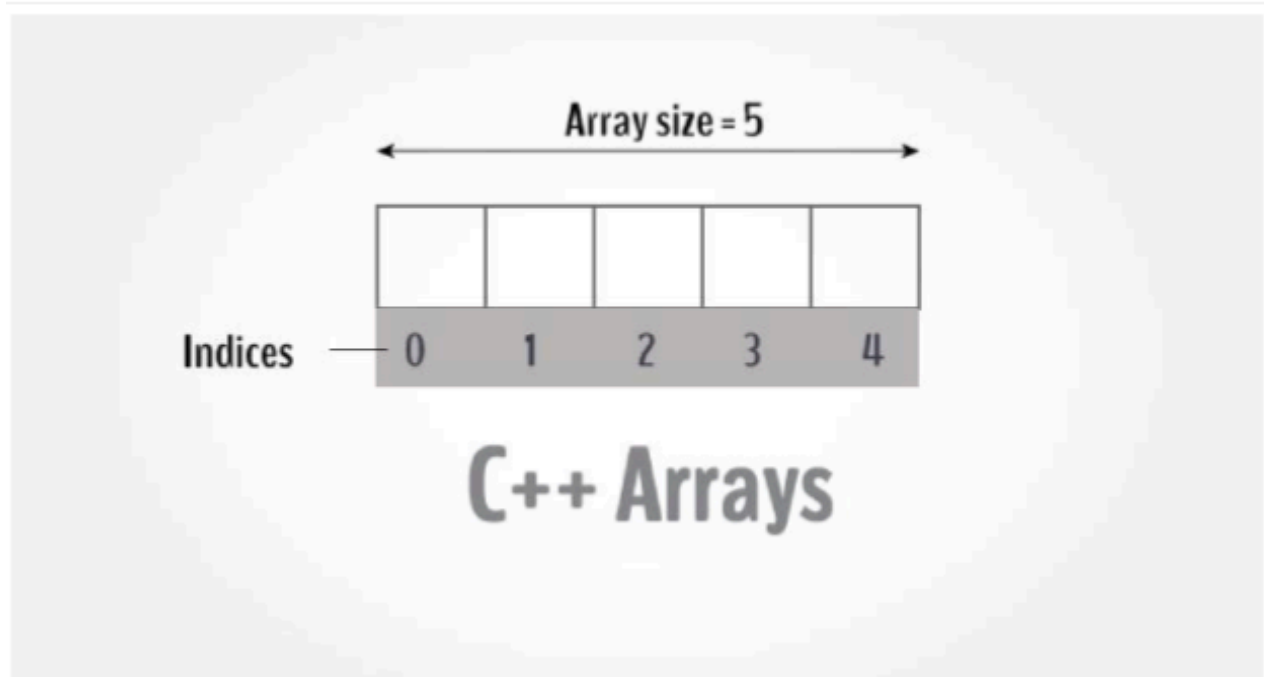
passes the value 60 to marks1, 70 to marks2 and lets the function use a default value of 75 for marks3.

The function call

```
average = result(60,70,80);
```

passes the value 80 to marks3.

C++ Arrays



In programming, one of the frequently arising problem is to handle numerous data of same type.

Consider this situation, you are taking a survey of 100 people and you have to store their age. To solve this problem in C++, you can create an integer array having 100 elements.

An array is a collection of data that holds fixed number of values of same type. For example:

```
int age[100];
```

Here, the `age` array can hold maximum of 100 elements of integer type.

The size and type of arrays cannot be changed after its declaration.

How to declare an array in C++?

```
dataType arrayName[arraySize];
```

For example,

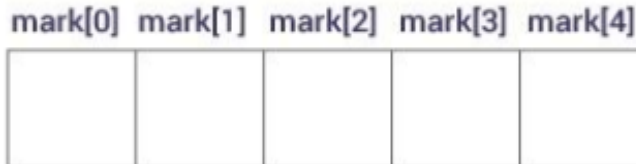
```
float mark[5];
```

Here, we declared an array, `mark`, of floating-point type and size 5. Meaning, it can hold 5 floating-point values.

Elements of an Array and How to access them?

You can access elements of an array by using indices.

Suppose you declared an array `mark` as above. The first element is `mark[0]`, second element is `mark[1]` and so on.



Few key notes:

- Arrays have 0 as the first index not 1. In this example, `mark[0]` is the first element.
- If the size of an array is n , to access the last element, $(n-1)$ index is used. In this example, `mark[4]` is the last element.
- Suppose the starting address of `mark[0]` is 2120d. Then, the next address, `a[1]`, will be 2124d, address of `a[2]` will be 2128d and so on. It's because the size of float is 4 bytes.

How to initialize an array in C++ programming?

It's possible to initialize an array during declaration. For example,

```
int mark[5] = {19, 10, 8, 17, 9};
```

Another method to initialize array during declaration:

```
int mark[] = {19, 10, 8, 17, 9};
```

mark[0]	mark[1]	mark[2]	mark[3]	mark[4]
19	10	8	17	9

Here,

```
mark[0] is equal to 19
```

```
mark[1] is equal to 10
```

```
mark[2] is equal to 8
```

```
mark[3] is equal to 17
```

```
mark[4] is equal to 9
```

How to insert and print array elements?

```
1. int mark[5] = {19, 10, 8, 17,  
2. 9}  
3.
```

```

    // change 4th element to
    9
4. mark[3]    =
5. 9;
6.
    // take input from the user and insert in third element
7. cin        >>
8. mark[2];
9.
10.
    // take input from the user and insert in (i+1)th element
11. cin        >>
12. mark[i];
13.
    // print first element of the array
14. cout        <<
15. mark[0];
16.
    // print ith element of the
    array
17. cout        >>
18. mark[i-1];

```

Example: C++ Array

C++ program to store and calculate the sum of 5 numbers entered by the user using arrays.

```

1.  #include
2.  <iostream>    using    ;
3.  namespace std
4.
5.  int main()
6.  {      int numbers[5], sum =
7.  0;      cout << "Enter 5
8.  numbers: ";
9.
10.         // Storing 5 number entered by user in an array
    .      // Finding the sum of numbers entered
11.  for (int i = 0; i < 5; ++i)
    .      {      cin >>
12.  numbers[i];      sum
    .      += numbers[i];
13.      }
    .
14.      cout << "Sum = " << sum <<
    .      endl;
15.
    .      return 0;
16.  }

```

```
.
17
.
18
.
19
.
20
.
```

Output

```
Enter 5 numbers: 3
4
5
4
2
Sum = 18
```

Things to remember when working with arrays in C++

Suppose you declared an array of 10 elements. Let's say,

```
int testArray[10];
```

You can use the array members from `testArray[0]` to `testArray[9]`.

If you try to access array elements outside of its bound, let's say `testArray[14]`, the compiler may not show any error. However, this may cause unexpected output (undefined behavior).

C++ Multidimensional Arrays

In C++, you can create an [array](#) of an array known as multi-dimensional array. For example:

```
int x[3][4];
```

Here, `x` is a two dimensional array. It can hold a maximum of 12 elements.

You can think this array as table with 3 rows and each row has 4 columns as shown below.

	Column 1	Column 2	Column 3	Column 4
Row 1	<code>x[0][0]</code>	<code>x[0][1]</code>	<code>x[0][2]</code>	<code>x[0][3]</code>
Row 2	<code>x[1][0]</code>	<code>x[1][1]</code>	<code>x[1][2]</code>	<code>x[1][3]</code>
Row 3	<code>x[2][0]</code>	<code>x[2][1]</code>	<code>x[2][2]</code>	<code>x[2][3]</code>

Multidimensional Array Initialisation

You can initialise a multidimensional array in more than one way.

Initialisation of two dimensional array

```
int test[2][3] = {2, 4, -5, 9, 0, 9};
```

Better way to initialise this array with same array elements as above.

```
int test[2][3] = { {2, 4, 5}, {9, 0, 0}};
```

Example 1: Two Dimensional Array

C++ Program to display all elements of an initialised two dimensional array.

```
1. #include
2. <iostream> using ;
3. namespace std
4.
5.
6.
7.
8.
```

```

9.  int main()
10 {
    .   int test[3][2] =
11     {
    .       {2, -5},
12         {4, 0},
    .       {9, 1}
13     } ;
    .
14     // Accessing two dimensional array using
    .   // nested for loops
15     for(int i = 0; i < 3; ++i)
16     {
    .         for(int j = 0; j < 2; ++j)
17         {
    .             cout<< "test[" << i << "][" << j << "] = " << test[i][j] <<
18             endl;
    .         }
19     }
    .
20     return 0;
    .
21
    .
22
    .
23
    .
24 }

```

Output

```

test[0][0] = 2
test[0][1] = -5
test[1][0] = 4
test[1][1] = 0
test[2][0] = 9
test[2][1] = 1

```

Example 2: Two Dimensional Array

C++ Program to store temperature of two different cities for a week and display it.

```

1.  #include
2.  <iostream> using ;
3.  namespace std
4.
5.

```

```

6.  const int CITY =
7.  2; const int WEEK
8.  = 7;
9.
10. int main()
11. {
12.     [CITY][WEEK];
13.     int
14.     temperature
15.     cout << "Enter all temperature for a week of first city and then
16.     second city. \n";
17.     // Inserting the values into the temperature array
18.     for (int i = 0; i < CITY; ++i)
19.     {
20.         for(int j = 0; j < WEEK; ++j)
21.         {
22.             cout << "City " << i + 1 << ", Day " << j + 1 << " : ";
23.             cin >> temperature[i][j];
24.         }
25.     }
26.
27.     cout << "\n\nDisplaying
28.     Values:\n ";
29.
30.     // Accessing the values from the temperature array
31.     for (int i = 0; i < CITY; ++i)
32.     {
33.         for(int j = 0; j < WEEK; ++j)
34.         {
35.             cout << "City " << i + 1 << ", Day " << j + 1 << " = " <<
36.             temperature[i][j] << endl;
37.         }
38.     }
39.
40.     return
41.     0; }

```

Output

```

Enter all temperature for a week of first city and then second city.
City 1, Day 1 : 32
City 1, Day 2 : 33
City 1, Day 3 : 32
City 1, Day 4 : 34
City 1, Day 5 : 35
City 1, Day 6 : 36
City 1, Day 7 : 38
City 2, Day 1 : 23

```

```

City 2, Day 2 : 24
City 2, Day 3 : 26
City 2, Day 4 : 22
City 2, Day 5 : 29
City 2, Day 6 : 27
City 2, Day 7 : 23

```

Displaying Values:

```

City 1, Day 1 = 32
City 1, Day 2 = 33
City 1, Day 3 = 32
City 1, Day 4 = 34
City 1, Day 5 = 35
City 1, Day 6 = 36
City 1, Day 7 = 38
City 2, Day 1 = 23
City 2, Day 2 = 24
City 2, Day 3 = 26
City 2, Day 4 = 22
City 2, Day 5 = 29
City 2, Day 6 = 27
City 2, Day 7 = 23

```

Global variable and local variable

Local Variable: A variable declared within the body of a function will be evaluated only within the function. The portion of the program in which a variable is retained in memory is known as the **scope of the variable**. The scope of the local variable is a function where it is defined. A variable may be local to function or compound statement.

Global variable: A variable that is declared outside any function is known as a global variable. The scope of such a variable extends till the end of the program. These variables are available to all functions which follow their declaration. So it should be defined at the beginning, before any function is defined.

Unary scope resolution operator (::) It is possible to declare local and global variables of the same name. C++ provides the **unary scope resolution operator (::)** to access a global variable when a local variable of the same name is in scope. A global variable can be accessed directly without the unary scope resolution operator if the name of the global variable is not the same as that of a local variable in scope.

Lab Tasks

1. Create and Implement a program in C++ to print areas of circle, rectangle, triangle, and square check using Functions.
2. Create and Implement a program in C++ to generate the multiplication table of a number (entered by the user) using Functions.

3. Create and Implement a program in C++ that will display the calculator menu. The program will prompt the user to choose the operation choice (from 1 to 5). Then it asks the user to input two integer values for the calculation. See the sample below. MENU 1. Add 2. Subtract 3. Multiply 4. Divide 5. Modulus Enter your choice: 1 Enter your two numbers: 12 15 Result: 27 Continue? y The program also asks the user to decide whether he/she wants to continue the operation. If he/she inputs 'y', the program will prompt the user to choose the operation gain. Otherwise, the program will terminate.
4. Write a C++ Program to Calculate Average of Numbers Using Arrays. • This program takes n number of element from user (where, n is specified by user), stores data in an array and calculates the average of those numbers.
5. Write C++ Program to Calculate Standard Deviation.
 - a. This program calculates the standard deviation of a individual series using arrays. Visit this page to learn about Standard Deviation.
6. Write a C++ Program to Add Two Matrix Using Multi-dimensional Arrays.
 - a. In this program, user is asked to entered the number of rows r and columns c. The value of r and c should be less than 100 in this program.
 - b. The user is asked to enter elements of two matrices (of order r*c).
 - c. Then, the program adds these two matrices, saves it in another matrix (twodimensional array) and displays it on the screen.

Conclusion

Lab Instructor

Kaynat Rana